جامعة الأميرة نورة بنت عبدالرحمن
Princess Nourah bint Abdulrahman University

DRIVER DROWSINESS DETECTION

Graduation Project 2

Presented by

Fay Almutairi

Sarah Albassam

Renad Alrashdi

Rahaf Alhuwati

Shahad Otay

Supervised By

Dr. Najwan Abdel Samee

Introduction

- Driver drowsiness is a leading cause of car accidents.

  It affects focus, slows reaction time, and often goes unnoticed.

- Every year, many crashes happen because drivers fall asleep while driving..

  There is a clear need for a system to detect drowsiness early and warn.

- Our project offers a simple and smart solution to help prevent these accidents.

  It uses a camera and AI to monitor the driver and alert them if they appear drowsy.

Problem Statement

Driver Drowsiness is a Risk why?

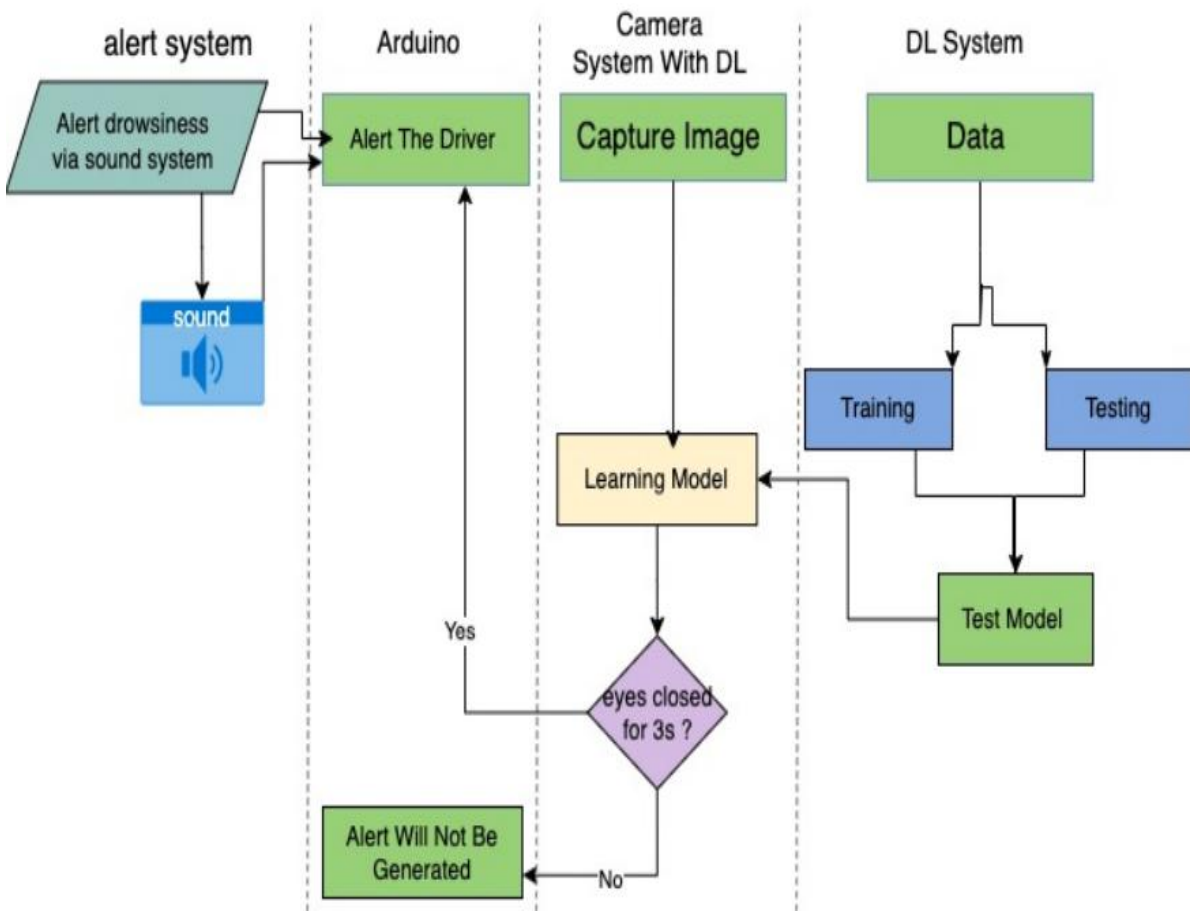
- 20% of drivers in Saudi Arabia admit to falling asleep while driving.
- 10% have been involved in accidents or close call due to drowsiness.
- **There is a Missing for a Warning Systems:** Many cars don’t have systems to alert drivers when they are getting sleepy.
- **A Solution is Needed:** We need a system that detects eye closure and warns the driver before an accident happens.

Proposed Solution

- The system uses Integrating AI into a driver drowsiness detection system can make roads safer by monitoring drivers in real time.
- a high-resolution Nicla Vision camera to detect signs such as closed eyes. If drowsiness is detected, the system triggers an immediate sound alert, warning the driver to be focused .
- This affordable and compact design makes it easy to implement and offers a reliable way to reduce accidents caused by fatigue.

System Architecture

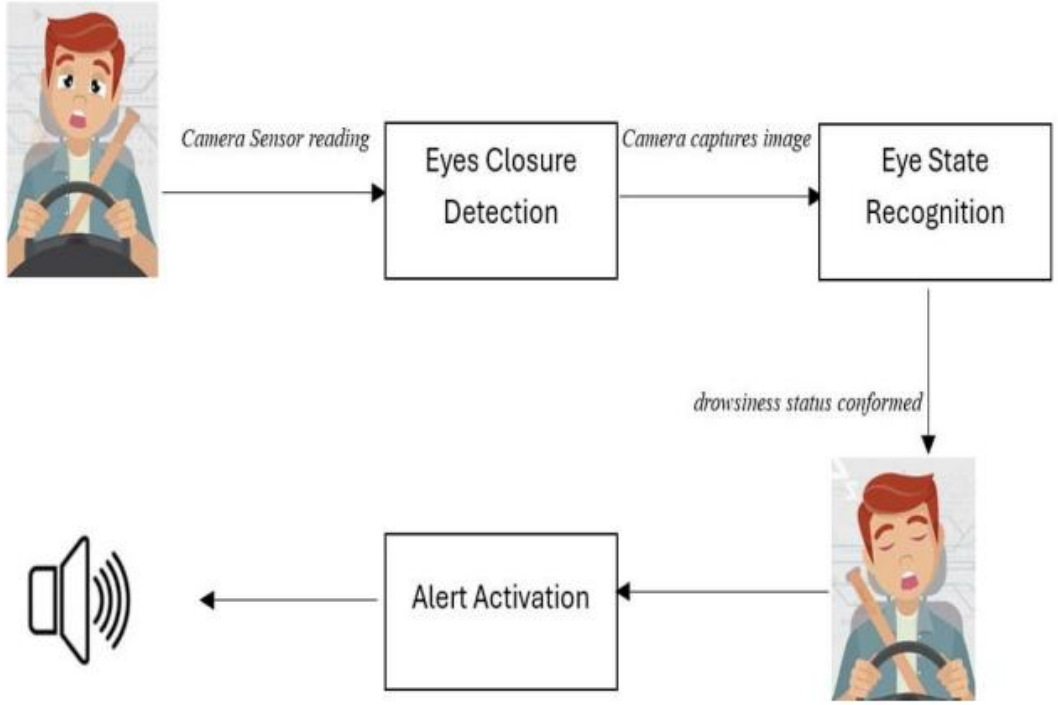
- **System Architecture:**
- **Nicla Vision Camera:** Captures images of the driver's eyes.
- **AI Algorithms:** AI algorithms are used to analyze and classify eye closure duration.
- **Alert System:**
  - If eyes remain closed for more than 3 seconds, the system classifies the driver as drowsy.
  - No alert if no drowsiness is detected.
- **Audio Alert:** A buzzer sounds to prompt the driver.



System Prototype Modules

The system is composed of three primary modules:

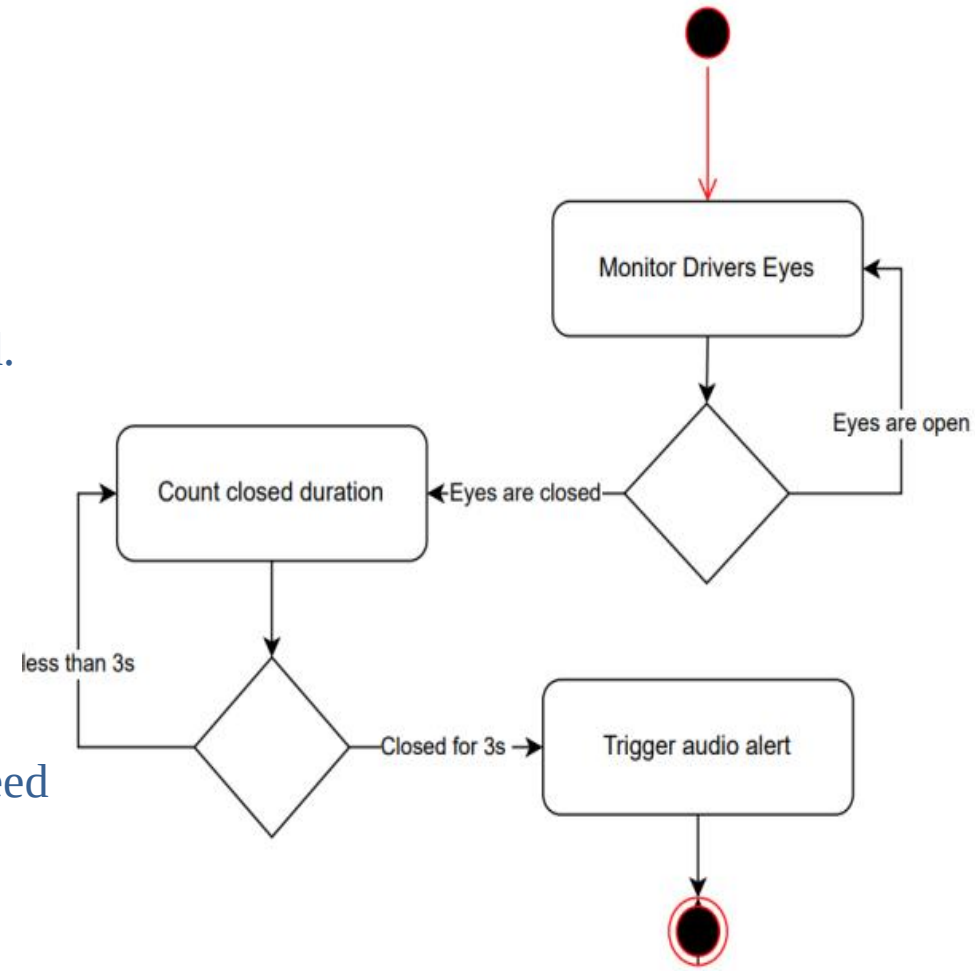
- **Eyes Closure Detection:** Uses the Nicla Vision camera to detect signs of drowsiness
- **Eye State Recognition:** Confirms signs of drowsiness by checking eye closure
- **Alert Activation:** Triggers an audio alert if drowsiness is confirmed.



Activity Diagram

The flow of the activities in the activity diagram includes the following:

- **Monitor Driver's Eyes:** Continuously track the driver's eyes to check if they are open or closed.
- **If Driver's Eyes are closed:** Start counting how long the eyes stay closed.
- **If Open:** Reset the counter and continue monitoring.
- **Duration Check:**
- **Less than 3 seconds:** If eyes are closed for less than 3 seconds, return to monitoring.
- **More than 3 seconds:** If eyes remain closed for 3 seconds or more, proceed to trigger alerts.
- **Trigger Alerts:** Activate an audio alert first to warn the driver.



Implementation Requirements

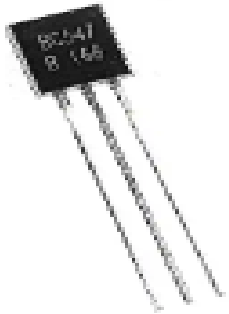
Hardware Requirements

Arduino Nicla Vision

Breadboard

Jumper Wires

Buzzer



NPN Transistor

Resistor

Software Requirements

Arduino IDE

TensorFlow

Edge Impulse

Visual Studio Code

Visual Studio Code Workflow

- To implement the **drowsiness detection model**, Visual Studio Code was used as the primary development environment. This setup allowed efficient integration of **Python-based** machine learning frameworks and **Arduino** support for deploying the model to the **Nicla Vision board**. After configuring the necessary extensions and dependencies, the process of **building**, **training**, and **evaluating** the model was carried out within this environment.

Main Stages Of Model Development

- **Dataset**

  To learn the model how to detect drowsiness, we used a dataset from Kaggle called Driver Drowsiness Dataset (DDD). It contains more than 41,790 face images, each labeled as either Drowsy or Non-Drowsy. These images were taken from real driving videos.

- **Data Preparation**

  Before training the model, we needed to prepare the data properly:
  Split the dataset:
  80% for training (to teach the model)
  20% for testing (to check the model later)

  All images were **resized** and **normalized** to match the input size needed for the MobileNet model.
  We used **data augmentation** (like rotating, flipping, and zooming the images) to help the model become more accurate and work well with different faces.

Main Stages Of Model Development

- **Model Training**

We built the model using **TensorFlow** and **Keras**, two popular tools for AI. We chose MobileNet because: It works well on small devices like the Nicla Vision It’s fast and doesn’t use much memory We used **transfer learning**, which means we started with a **pre-trained model** and adjusted it using our drowsiness dataset. This made training faster and more accurate, even with limited resources.

Main Stages Of Model Development

- **Learning Parameters:**

- **Batch Size:** 32
  Balanced training speed and generalization.
- **Learning Rate:** 1e-5
  Ensured stable and precise weight updates.
- **Validation Frequency:** Every epoch
  Enabled close monitoring and early overfitting detection.
- **Dropout Rates:** 0.5 and 0.4
  Improved generalization by preventing overfitting.
- **Epochs Tested:** 10 to 100
  Best performance at 50 epochs with high accuracy and stability.

Learning Parameters	Values
Batch Size	32
Initial Learning Rate	1e-5
Validation Frequency	1
Dropout	0.5, 0.4
No. of Epochs	10, 20, 30, 40, 50, and 100

Main Stages Of Model Development

- **Model Evaluation**

  After training, we tested the model using the 20% test set.

  We evaluated its performance using:

  Accuracy – How often it gives the correct result

  Precision – How good is it at detecting true drowsy cases

  Recall – How many actual drowsy cases it catch

  Confusion Matrix – A chart showing the model's correct and incorrect guesses

Process Of Building The Model

- **1- We used VS with a Python virtual environment to build and train our AI model, set up folders, and file paths**

Key libraries:

- **TensorFlow** – For training the MobileNet model
- **OpenCV** – For image processing
- **NumPy** – For numerical operations
- **matplotlib** – For visualizing training results
- **pandas** – For handling data
- **scikit-learn** – For evaluating model performance

```python
import os
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
import pandas as pd
from sklearn.metrics import confusion_matrix, classification_report, roc_auc_score, matthews_corrcoef, f1_score, roc_curve
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import Dense, Dropout, GlobalAveragePooling2D
from tensorflow.keras.models import Model, load_model
from tensorflow.keras.regularizers import l2
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.callbacks import ReduceLROnPlateau
from sklearn.model_selection import train_test_split
```

[1] Python

Configuration & Directories Setup

```python
DATA_DIR = os.path.join(os.getcwd(), "Driver Drowsiness Dataset (DDD)")
DROWSY_DIR = os.path.join(DATA_DIR, 'Drowsy')
NON_DROWSY_DIR = os.path.join(DATA_DIR, 'Non Drowsy')
OUTPUT_DIR = "prepared_data"

# Validate dataset availability
assert os.path.exists(DROWSY_DIR), "Drowsy folder not found"
assert os.path.exists(NON_DROWSY_DIR), "Non-drowsy folder not found"

# Create dataset partitions
for split in ["train", "val", "test"]:
    for category in ["drowsy", "non_drowsy"]:
        os.makedirs(os.path.join(OUTPUT_DIR, split, category), exist_ok=True)
```

[18] Python

Process Of Building The Model

➢ 2- Data Preprocessing, Splitting, and Augmentation

The images are resized, split into training and testing sets, and enhanced with transformations like rotation and zoom.

Data Preprocessing & Splitting

```python
def split_and_copy(source_dir, target_dirs, split_ratios):
    filenames = os.listdir(source_dir)
  train, test = train_test_split(filenames, test_size=split_ratios[1] + split_ratios[2], random_state=42)
   val, test = train_test_split(test, test_size=split_ratios[2] / (split_ratios[1] + split_ratios[2]), random_state=42)

    for split_name, files in zip(["train", "val", "test"], [train, val, test]):
        category_name = os.path.basename(source_dir).lower().replace(" ", "_")
       category_path = os.path.join(target_dirs[split_name], category_name)
        os.makedirs(category_path, exist_ok=True)
        for file in files:
            shutil.copy(os.path.join(source_dir, file), os.path.join(category_path, file))

split_and_copy(DROWSY_DIR, {"train": os.path.join(OUTPUT_DIR, "train"), "val": os.path.join(OUTPUT_DIR, "val"), "test": os.path.join(OUTPUT_DIR, "test")}, [0.7, 0.2, 0.1])
split_and_copy(NON_DROWSY_DIR, {"train": os.path.join(OUTPUT_DIR, "train"), "val": os.path.join(OUTPUT_DIR, "val"), "test": os.path.join(OUTPUT_DIR, "test")}, [0.7, 0.2, 0.1])
```

Data Augmentation & Generators

```python
train_datagen = ImageDataGenerator(
    rescale=1.0/255.0,
    rotation_range=30,
    zoom_range=0.3,
    width_shift_range=0.3,
    height_shift_range=0.3,
    shear_range=0.3,
    horizontal_flip=True,
    fill_mode="nearest"
)

val_datagen = ImageDataGenerator(rescale=1.0/255.0)

train_generator = train_datagen.flow_from_directory(os.path.join(OUTPUT_DIR, "train"), target_size=(96, 96), batch_size=32, class_mode='binary')
val_generator = val_datagen.flow_from_directory(os.path.join(OUTPUT_DIR, "val"), target_size=(96, 96), batch_size=32, class_mode='binary')
```

Process Of Building The Model

➢ 3- Based Model Architecture

Defines the structure of the AI model, using MobileNetV2 for efficient drowsiness detection on small devices.

Model Architecture

```python
base_model = MobileNetV2(weights='imagenet', include_top=False, input_shape=(96, 96, 3), alpha=0.35)
base_model.trainable = False

x = GlobalAveragePooling2D()(base_model.output)
x = Dropout(0.5)(x)
x = Dense(64, activation='relu', kernel_regularizer=l2(0.01))(x)
x = Dropout(0.4)(x)
output = Dense(1, activation='sigmoid')(x)

model = Model(inputs=base_model.input, outputs=output)
```

Python

Process Of Building The Model

➢ 4- Evaluation Function for Model Performance Metrics and Confusion Matrix

Contains functions to measure model accuracy, precision and to generate a confusion matrix.

Evaluation Function

```python
eval_results = []

def evaluate_model(model, test_generator, epoch):
    y_true = test_generator.classes
    y_pred_prob = model.predict(test_generator)
    y_pred = (y_pred_prob > 0.5).astype(int).flatten()

    # Confusion matrix
    cm = confusion_matrix(y_true, y_pred)
    tn, fp, fn, tp = cm.ravel()

    accuracy = (tp + tn) / (tp + tn + fp + fn)
    sensitivity = tp / (tp + fn) if (tp + fn) != 0 else 0  # Recall
    specificity = tn / (tn + fp) if (tn + fp) != 0 else 0
    precision = tp / (tp + fp) if (tp + fp) != 0 else 0
    fnr = fn / (fn + tp) if (fn + tp) != 0 else 0
    fpr = fp / (fp + tn) if (fp + tn) != 0 else 0
    auc = roc_auc_score(y_true, y_pred_prob)
    mcc = matthews_corrcoef(y_true, y_pred)
    f1 = f1_score(y_true, y_pred)

    eval_results.append({
        "Epoch": epoch,
        "TP": tp, "TN": tn, "FP": fp, "FN": fn,
        "Accuracy": accuracy,
        "Sensitivity": sensitivity,
        "Specificity": specificity,
        "Precision": precision,
        "False Negative Rate": fnr,
        "False Positive Rate": fpr,
        "AUC": auc,
        "Matthews CorrCoef": mcc,
        "F1-Score": f1
    })
```

```python
# Plot and Save Confusion Matrix
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Non-Drowsy', 'Drowsy'], yticklabels=['Non-Drowsy', 'Drowsy'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title(f'Confusion Matrix - Epoch {epoch}')
plt.savefig(f"confusion_matrix_{epoch}_epochs.png")
plt.show()

# Plot and Save ROC Curve
fpr_curve, tpr_curve, _ = roc_curve(y_true, y_pred_prob)
plt.figure(figsize=(6, 4))
plt.plot(fpr_curve, tpr_curve, label=f'AUC = {auc:.2f}')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title(f'ROC Curve - Epoch {epoch}')
plt.legend(loc='lower right')
plt.grid(True)
plt.savefig(f"roc_curve_{epoch}_epochs.png")
plt.show()

print("Classification Report:")
print(classification_report(y_true, y_pred, target_names=['Non-Drowsy', 'Drowsy']))
```

Python

Process Of Building The Model

➢ 5- Model Conversion to TensorFlow Lite (TFLite) with Quantization

Converts the trained model to a lightweight version (TFLite) so it can run on edge devices like Nicla Vision.

Convert Model to TensorFlow Lite (TFLite) with Quantization

```python
def convert_and_save_tflite_model(model, output_path="/Users/saraalbassam23/Desktop/gp-2-model/drowsiness_detection_model_quantized_fixed.tflite"):
    def representative_dataset():
        for _ in range(100):
            data = np.random.rand(1, 96, 96, 3).astype(np.float32)
            yield [data]

    converter = tf.lite.TFLiteConverter.from_keras_model(model)
    converter.optimizations = [tf.lite.Optimize.DEFAULT]
    converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
    converter.inference_input_type = tf.uint8
    converter.inference_output_type = tf.uint8
    converter.representative_dataset = representative_dataset

    tflite_quantized_model = converter.convert()
    with open(output_path, "wb") as f:
        f.write(tflite_quantized_model)

    print("TFLite model conversion and quantization complete.")
```

[6] ✓ 0.0s Python

Process Of Building The Model

➢ 6- Model Compilation, Training, and Export to TensorFlow Lite

Shows the full training process and how the final model is exported in TFLite format.

Model Compilation & Training

```python
model.compile(optimizer=tf.keras.optimizers.Sara(learning_rate=0.0001), loss='binary_crossentropy', metrics=['accuracy'])

history = model.fit(train_generator, validation_data=val_generator, epochs=10,
                    steps_per_epoch=train_generator.samples // train_generator.batch_size,
                    validation_steps=val_generator.samples // val_generator.batch_size)
```

[33] Python

```
Epoch 1/10
914/914 ━━━━━━━━━━━━━━━━━━━━ 130s 139ms/step - accuracy: 0.6190 - loss: 0.6674 - val_accuracy: 0.8767 - val_loss: 0.3960
Epoch 2/10
  1/914 ━━━━━━━━━━━━━━━━━━━━ 18s 21ms/step - accuracy: 0.7500 - loss: 0.5326
c:\Users\cannonball\AppData\Local\Programs\Python\Python311\Lib\site-packages\keras\src\trainers\epoch_iterator.py:107: UserWarning: Your input ran out of data; interrupting training. Make s
  self._interrupted_warning()
914/914 ━━━━━━━━━━━━━━━━━━━━ 14s 15ms/step - accuracy: 0.7500 - loss: 0.5326 - val_accuracy: 0.8766 - val_loss: 0.3959
Epoch 3/10
914/914 ━━━━━━━━━━━━━━━━━━━━ 85s 93ms/step - accuracy: 0.8001 - loss: 0.4353 - val_accuracy: 0.9351 - val_loss: 0.2715
Epoch 4/10
914/914 ━━━━━━━━━━━━━━━━━━━━ 12s 14ms/step - accuracy: 0.7500 - loss: 0.4329 - val_accuracy: 0.9345 - val_loss: 0.2720
Epoch 5/10
914/914 ━━━━━━━━━━━━━━━━━━━━ 83s 91ms/step - accuracy: 0.8481 - loss: 0.3549 - val_accuracy: 0.9437 - val_loss: 0.2035
Epoch 6/10
914/914 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - accuracy: 0.7500 - loss: 0.4225 - val_accuracy: 0.9434 - val_loss: 0.2037
Epoch 7/10
914/914 ━━━━━━━━━━━━━━━━━━━━ 81s 89ms/step - accuracy: 0.8821 - loss: 0.2924 - val_accuracy: 0.9691 - val_loss: 0.1502
Epoch 8/10
914/914 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.9688 - loss: 0.2012 - val_accuracy: 0.9695 - val_loss: 0.1501
Epoch 9/10
914/914 ━━━━━━━━━━━━━━━━━━━━ 84s 92ms/step - accuracy: 0.9012 - loss: 0.2491 - val_accuracy: 0.9761 - val_loss: 0.1217
Epoch 10/10
914/914 ━━━━━━━━━━━━━━━━━━━━ 12s 14ms/step - accuracy: 0.9688 - loss: 0.1894 - val_accuracy: 0.9758 - val_loss: 0.1222
```

Process Of Building The Model

➢ 7- Epoch Loop Execution and Training Results Output

Displays the training loop with performance stats such as accuracy and loss after each cycle (epoch).

```python
epochs = [10,20,30,40,50,100]

def train_save_results():
    model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5), loss='binary_crossentropy', metrics=['accuracy'])

    test_datagen = ImageDataGenerator(rescale=1.0/255.0)
    test_generator = test_datagen.flow_from_directory(
        os.path.join(OUTPUT_DIR, "test"),
        target_size=(96, 96),
        batch_size=32,
        class_mode='binary',
        shuffle=False
    )

    lr_scheduler = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3, min_lr=1e-7, verbose=1)

    for i, e in enumerate(epochs):
        model.fit(
            train_generator,
            validation_data=val_generator,
            epochs=e,
            steps_per_epoch=train_generator.samples // train_generator.batch_size,
            validation_steps=val_generator.samples // val_generator.batch_size,
            callbacks=[lr_scheduler]
        )
```

```python
            test_loss, test_acc = model.evaluate(test_generator)
            print(f"Test Accuracy after {e} epochs: {test_acc * 100:.2f}%")

            evaluate_model(model, test_generator, epoch=e)
            model.save(f"drowsiness_detection_model_{e}.h5")

            if i == len(epochs) - 1:
                convert_and_save_tflite_model(model)

                df_results = pd.DataFrame(eval_results)
                df_results.to_excel("evaluation_metrics.xlsx", index=False)
                print("Evaluation metrics saved to evaluation_metrics.xlsx")
```

[7] ✓ 0.0s Python

```python
train_save_results()
```

[14] Python

```
Found 4180 images belonging to 2 classes.
/Users/saraalbassam23/venv/lib/python3.9/site-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().__init__(**kwargs)`
  self._warn_if_super_not_called()
Epoch 1/10
914/914 ━━━━━━━━━━━━━━━━━━━━ 94s 101ms/step - accuracy: 0.5063 - loss: 2.1165 - val_accuracy: 0.5809 - val_loss: 1.7425 - learning_rate: 1.0000e-05
Epoch 2/10
```

⊗ 0 ⚠ 0 Cell 13 of 17

Edge Impulse Configuration

- **Edge Impulse** is an important tool in this project. We uploaded our trained AI model to **Edge Impulse**, which acts as a **bridge** to deploy it to the Nicla Vision board. This allows the model to run in real time on the device without needing the internet or cloud services and helps us detect drowsiness directly from the camera input.

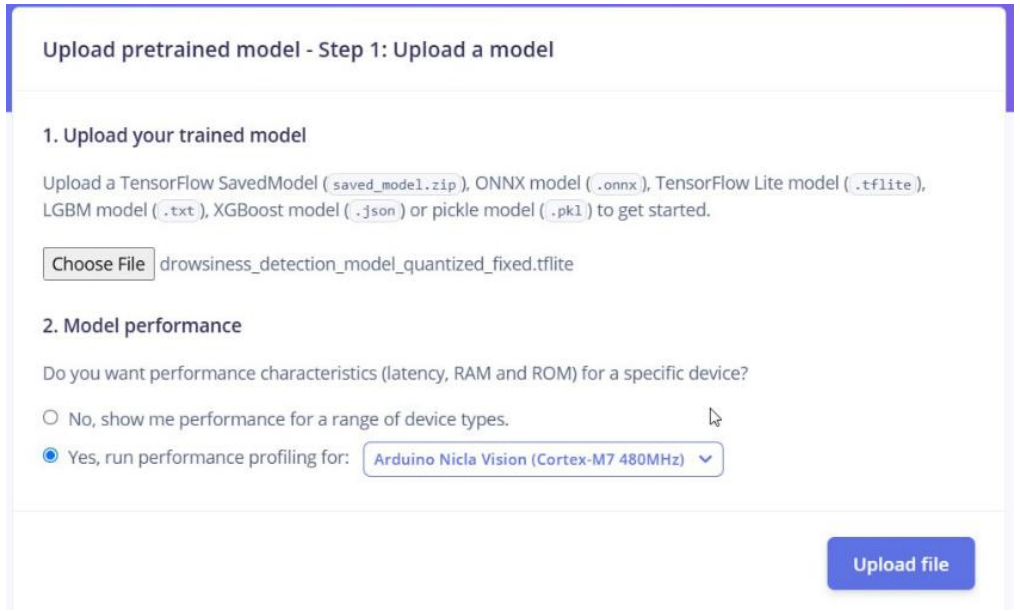
We used Edge Impulse in two main purposes:

- **A)** Deploy the model directly to the Nicla Vision camera, so it can detect drowsiness on its own using captured face images.
- **B)** Deploy the AI model as an Arduino library, so we could add code to trigger a buzzer alert when drowsiness is detected.

Edge Impulse Configuration

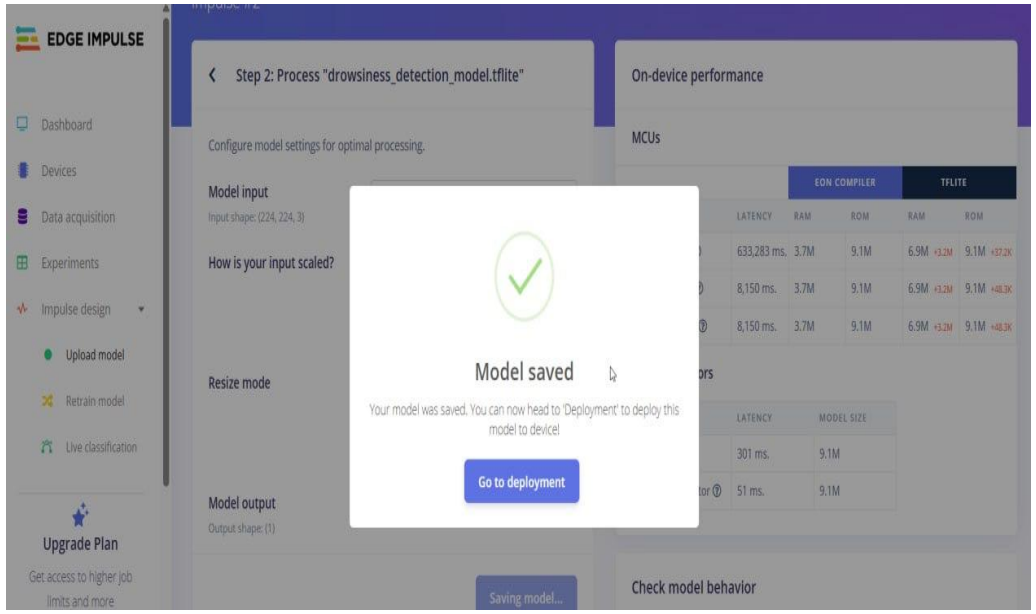
A) Uploading and Exporting the Model as an Arduino Library

➢ 1- Uploading the Model



Uploading our trained AI model to Edge Impulse for further configuration and deployment

➢ 2- Model saved to The edge



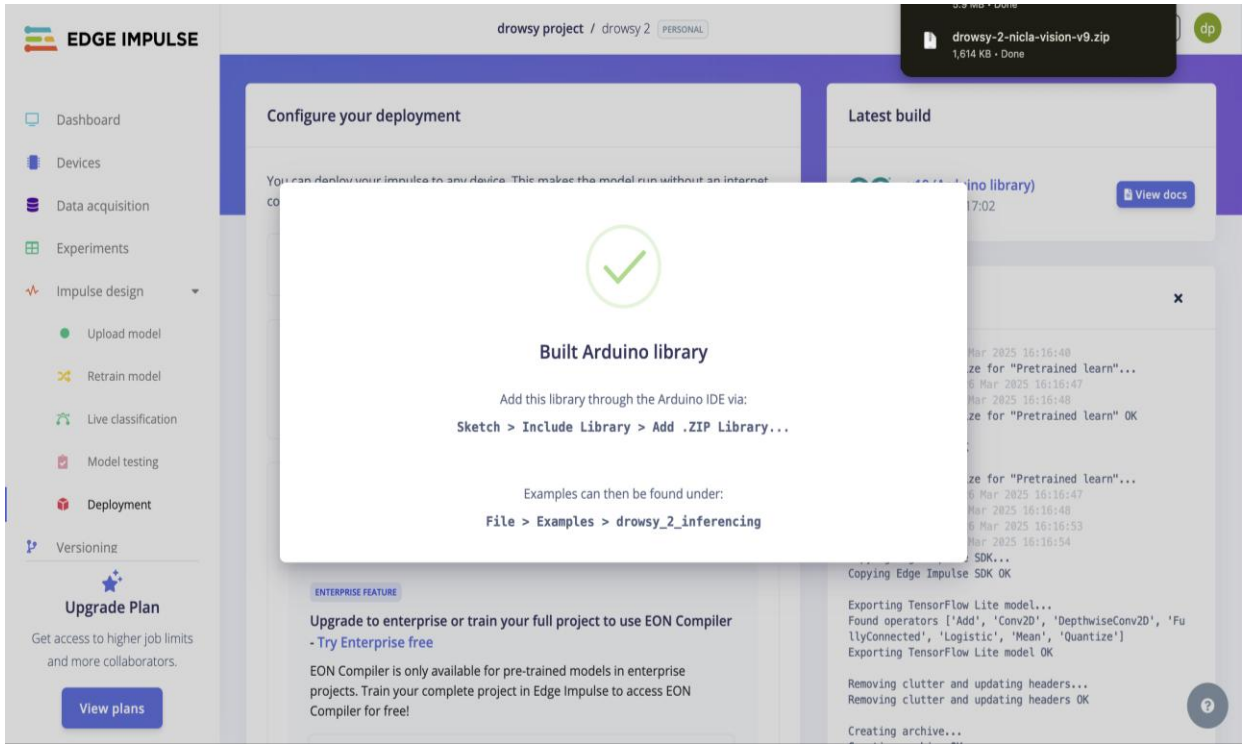
Impulse settings are saved after uploading the model.

Edge Impulse Configuration

A) Uploading and Exporting the Model as an Arduino Library

➢ **3- Saving the model as Arduino library**

- The model is exported as an Arduino library (.zip file) to use in the Arduino IDE.



Edge Impulse Configuration

B) Flashing the Model to Nicla Vision Using Edge Impulse CLI

➢ 1- Install the edge impulse CLI

- To run the model directly on the Nicla Vision board, we used the Edge Impulse CLI to connect our device and flash the model onto it.

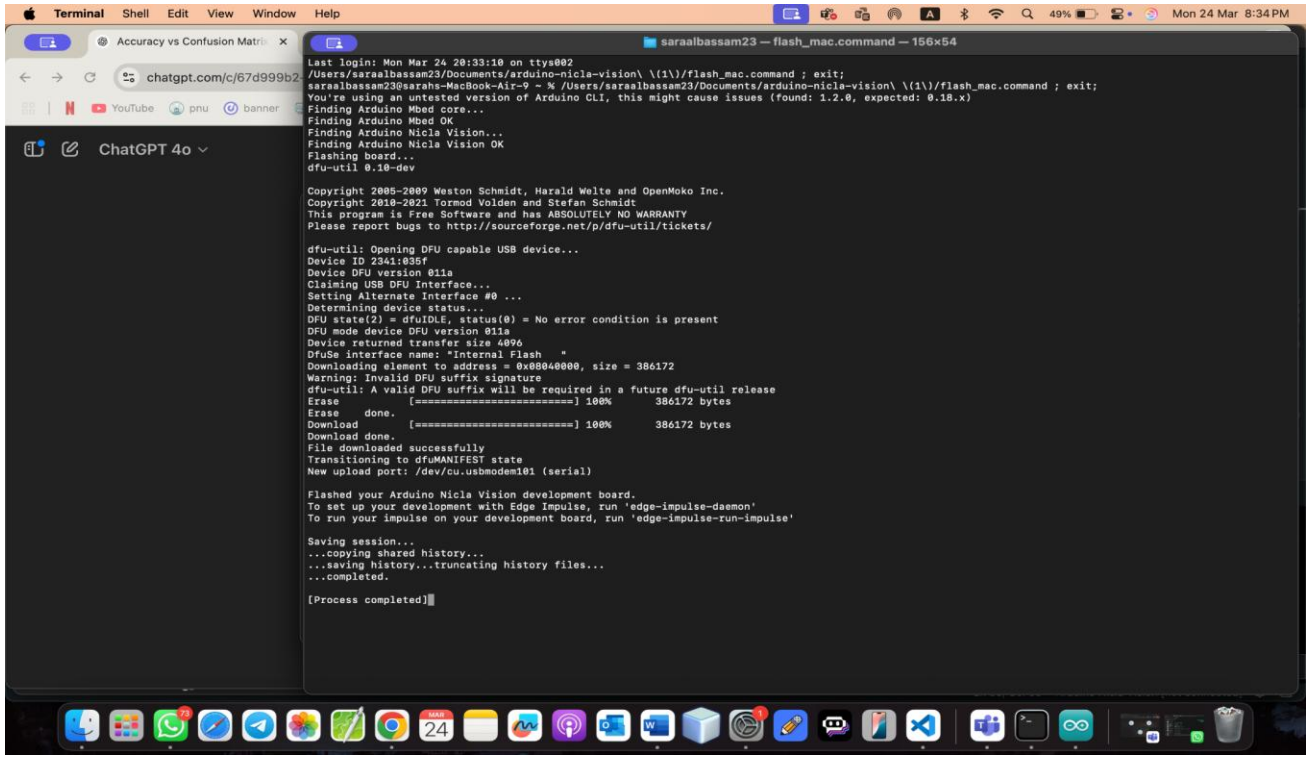
```
Last login: Mon Mar 24 22:10:06 on ttys002
saraalbassam23@sarahs-MacBook-Air-9 ~ % edge-impulse-daemon -version
Edge Impulse serial daemon v1.31.0
Endpoints:
    Websocket: wss://remote-mgmt.edgeimpulse.com
    API:       https://studio.edgeimpulse.com
    Ingestion: https://ingestion.edgeimpulse.com
```

Edge Impulse Configuration

B) Flashing the Model to Nicla Vision Using Edge Impulse CLI

➢ 2- Nicla Vision firmware

- Physically connected the Nicla Vision camera to our computer using a USB cable. Then, we downloaded the official Nicla Vision firmware from Edge Impulse. Once this process was completed, the firmware was successfully installed on the device

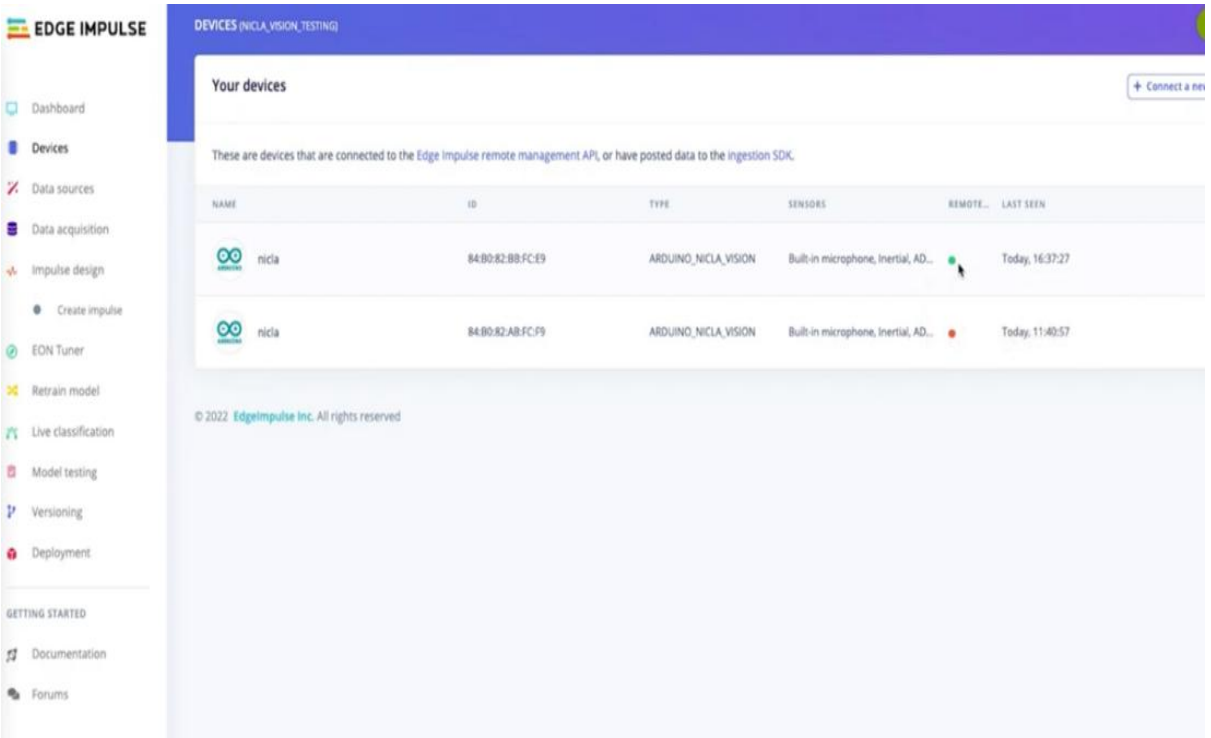


Edge Impulse Configuration

B) Flashing the Model to Nicla Vision Using Edge Impulse CLI

➢ 3- Camera connection to Edge Impulse

- To connect the camera to our Edge Impulse project, we opened a terminal and ran the command: *edge-impulse-daemon*
- We entered our Edge Impulse login credentials and then selected our project from the list. After that, the Nicla Vision camera appeared under the Devices tab in Edge Impulse Studio with a green indicator, confirming that it was connected and ready.

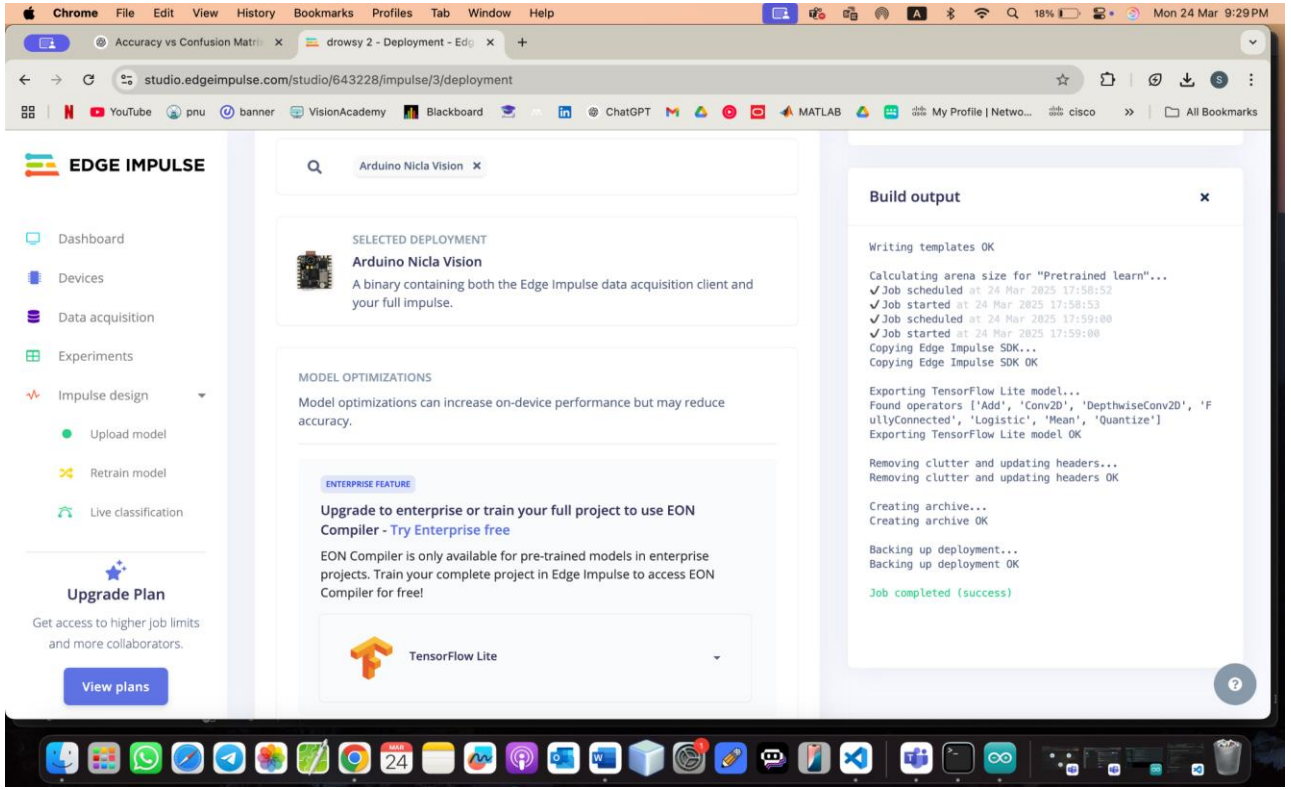


Edge Impulse Configuration

B) Flashing the Model to Nicla Vision Using Edge Impulse CLI

➢ 4- Convert the model format

- The final step was flashing the model to the Nicla Vision board. From within Edge Impulse, we built the model in a format compatible with Nicla Vision.



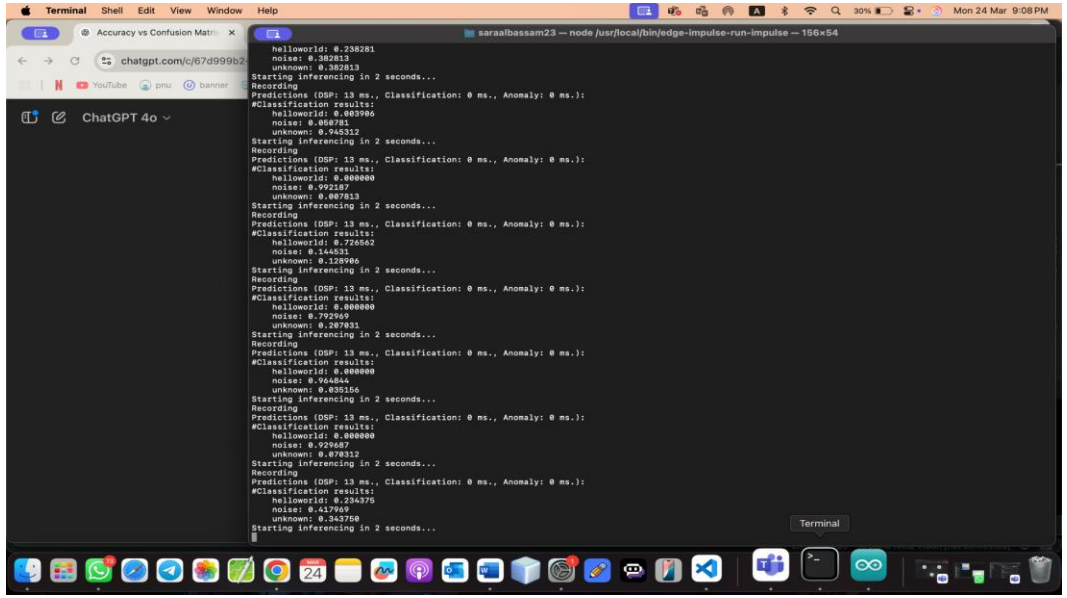
Edge Impulse Configuration

B) Flashing the Model to Nicla Vision Using Edge Impulse CLI

➢ 5- Running the AI Model

This command successfully uploaded the model to the device. The model runs directly on the Nicla Vision to detect drowsiness in real time

➢ 6- The output result



The device shows live prediction results (Drowsy or Not Drowsy) from the onboard camera.

Arduino IDE Configuration

- The software provides all the essential tools for writing, verifying, and uploading code to the hardware.
- We proceeded to install the necessary board support packages and libraries. Specifically, we added support for the Nicla Vision board and included the Edge Impulse Arduino library that we had previously exported.
- This allowed us to easily call our AI model within the Arduino environment and write the logic for real-time drowsiness detection.

Arduino IDE Configuration

1- Initial Buzzer Setup and Detection Parameters

```
#define BUZZER_PIN 3
int drowsyCount = 0;
const int thresholdFrames = 30;

```

Sets up the buzzer pin and defines how long the driver must appear drowsy before an alert is triggered.

2- Buzzer Pin Setup in setup() Function

```
Serial.println("Edge Impulse Inferencing Demo");
pinMode(BUZZER_PIN, OUTPUT);
digitalWrite(BUZZER_PIN, LOW);

```

Initializing the buzzer pin and ensuring it is off when the system starts.

Arduino IDE Configuration

- **3- Drowsiness Detection and Buzzer Activation Code**

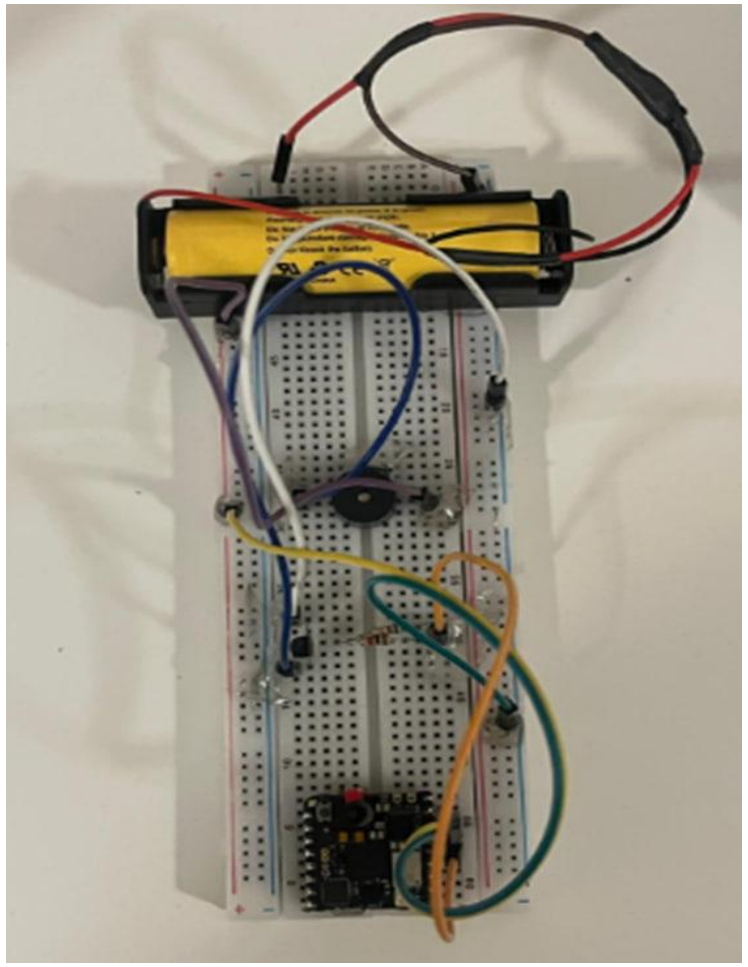
```
if (prediction == "drowsy") {
    drowsyCount++;
} else {
    drowsyCount = 0;
}

if (drowsyCount >= thresholdFrames) {
    ei_printf("Drowsiness detected! Beeping buzzer for 4 seconds.\r\n");
    digitalWrite(BUZZER_PIN, HIGH);
    delay(4000);
    digitalWrite(BUZZER_PIN, LOW);
    drowsyCount = 0;
}
```

The logic checks for drowsiness and triggers the buzzer if needed.

The Proposed Architecture

- **Final proposed architecture:**



Testing

Test Plan

- Our goal was to build a system that detects driver drowsiness in real time using a camera and alerts the driver with a sound.
- **How it Works:**
  - The system captures images and uses a MobileNetV2 deep learning model to classify whether the driver is Drowsy or Not Drowsy.
- **Model Training:**
  - **Dataset: 80%** for training, **20%** for testing
  - Trained using Visual Studio Code
  - Trained for 100 epochs to improve performance
- **Evaluation Metrics:**
  - Accuracy, Precision, Recall (Sensitivity)
  - Specificity, F1-Score
  - False Positive & Negative Rates
  - AUC and MCC (for balanced evaluation)

Test Plan

- To validate the system's behavior in real-world scenarios, four test cases were conducted as part of the testing process:

- **Case 1 – System Initialization and Model Uploading:**

Objective:

Make sure the system starts properly, and the model loads without issues.

Steps:

1. Connect the Nicla Vision camera
2. Load the trained model into the device

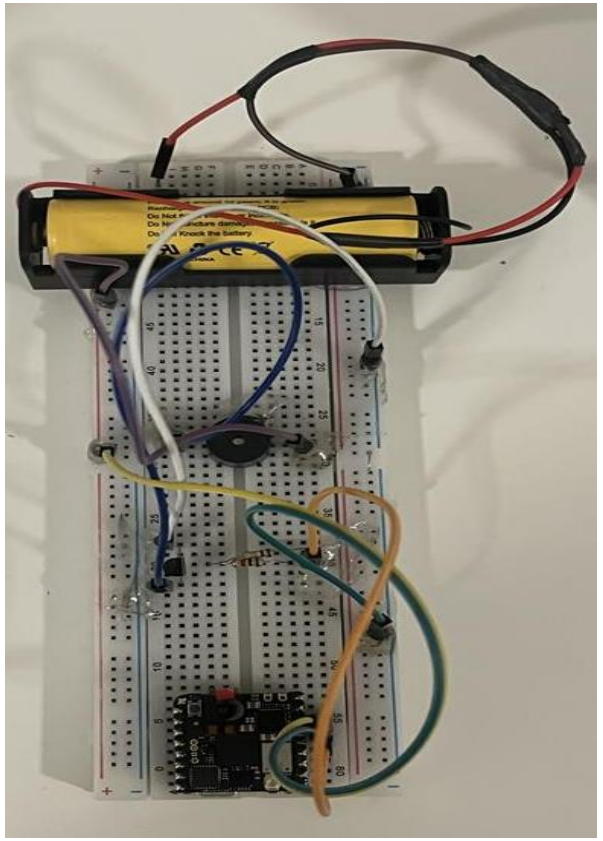
Expected Result:

No errors model is loaded successfully

Actual Result:

Model loaded with no issues

Status: Passed (1 trial)



Test Plan

- To validate the system's behavior in real-world scenarios, four test cases were conducted as part of the testing process:

- **Case 2 - Normal Eye Blinking**

Objective:

Ensure quick, natural eye blinks don’t trigger the alarm.

Steps:

1. Blink normally while facing the camera
2. Repeat for 10 trials
3. Monitor buzzer behavior

Expected Result:

The buzzer should not turn on during normal blinks

Actual Result:

The buzzer stayed off in 9 out of 10 trials

Status: Passed (10 trials)

Test Plan

- To validate the system's behavior in real-world scenarios, four test cases were conducted as part of the testing process:

- **Case 3 - Drowsiness Detection in Different Lighting Conditions**

Objective:

Test the system’s accuracy under both bright and dim lighting.

Steps:

1. Close eyes under each lighting condition (light, dim)
2. Observe
3. Monitor buzzer response

Expected Result:

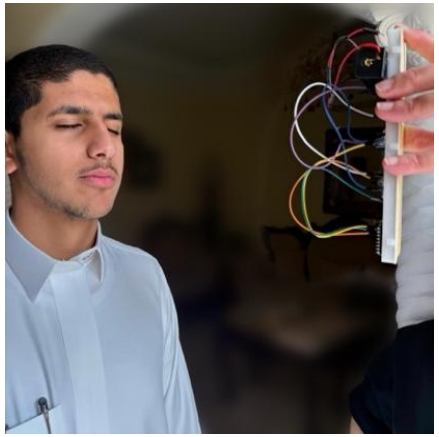
The buzzer should trigger in both conditions

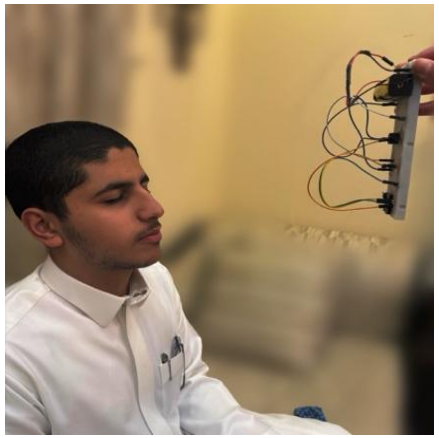
Actual Result:

Bright: 4/5 correct detections (80%)

Dim: 3/5 correct detections (60%)

Status: Passed (10 trials)





Test Plan

- To validate the system's behavior in real-world scenarios, four test cases were conducted as part of the testing process:

- **Case 4 – Detection When No Face in Frame**

Objective:

Ensure the system doesn’t falsely detect drowsiness when no face is present.

Steps:

1. Remove face from the camera view
2. Observe model output for 10 trials

Expected Result:

No detection, buzzer should remain off

Actual Result:

10/10 correct – no false alarms

Status: Passed (10 trials)

Test Result

- **To evaluate how well the model detects drowsiness, we used a set of performance metrics.**
- The model classifies images into:
  - **Drowsy**
  - **Not Drowsy**
- We measured how accurate and reliable these classifications are using the following:
  - **True Positive (TP):** Correctly predicted Drowsy
  - **True Negative (TN):** Correctly predicted Not Drowsy
  - **False Positive (FP):** Wrongly predicted Drowsy
  - **False Negative (FN):** Missed Drowsy case
- **Metric**

  Accuracy, Precision, Recall (Sensitivity), Specificity, F1-Score, False Positive Rate (FPR), False Negative Rate (FNR), Matthews Correlation Coefficient (MCC), Area Under the Curve (AUC)

Test Result

- **A confusion matrix** is a table that shows how well a classification model performs. It compares the actual results with the model's predictions.

- **It's useful:**
  - Helps visualize errors
  - Shows where the model is right or wrong
  - Used to calculate key metrics like accuracy, precision, recall, and F1-score

		Actual Values	
		Positive	Negative
Predicted Values	Positive	True Positive	False Positive
	Negative	False Negative	True Negative

Test Result

- **The Receiver Operating Characteristic (ROC) curve** shows how well a classification model can separate two classes:
  - **Drowsy**
  - **Not Drowsy**
- It plots:
  - **True Positive Rate (TPR)** = How many drowsy drivers were correctly detected
  - **False Positive Rate (FPR)** = How many not-drowsy drivers were wrongly flagged
- A key metric derived from the ROC curve is the **Area Under the Curve (AUC).**
  - An AUC of **1.0** represents a **perfect classifier**, while an AUC of **0.5** indicates performance equivalent **to random guessing**.
  - A model with an AUC closer to **1.0** is considered **highly effective**, whereas values near **0.5** suggest **poor classification ability**.

Binary Classification

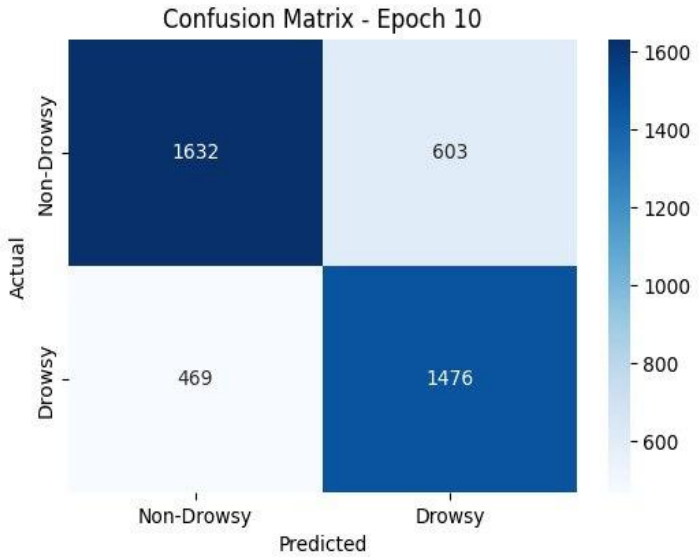
- We chose **Binary Classification** because our system needs to answer a simple, yet **critical question:**
  - **Is the driver drowsy or not?**

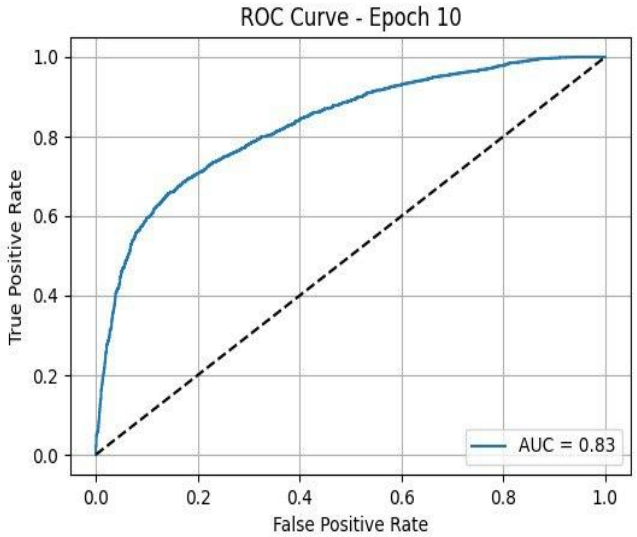
- The evaluation results of the drowsiness detection model across different training epochs are presented in the Table. The results demonstrate a clear improvement in the model's performance over time.

No of iterations	TP	TN	FP	FN	Accuracy	Sensitivity	Specificity	Precision	False Negative Rate	False Putative Rate	AUC	Matthews Correlation Coefficient	F1-score
10	1476	1632	603	469	74.35%	75.89%	73.02%	70.99%	24.11%	26.98%	0.82	0.4878	0.7335
20	1514	1858	377	431	80.67%	77.84%	83.13%	80.06%	22.16%	16.87%	0.89	0.6110	0.7893
30	1668	1948	287	277	86.51%	85.76%	87.16%	85.32%	14.24%	12.84%	0.93	0.7289	0.8553
40	1734	2008	227	211	89.52%	89.15%	89.84%	88.42%	10.85%	10.16%	0.96	0.7895	0.8878
50	1812	2070	165	133	92.87%	93.16%	92.62%	91.65%	6.84%	7.38%	0.97	0.8569	0.9240
100	1814	2143	92	131	94.67%	93.26%	95.88%	95.17%	6.74%	4.12%	0.98	0.8928	0.9420

Binary Classification

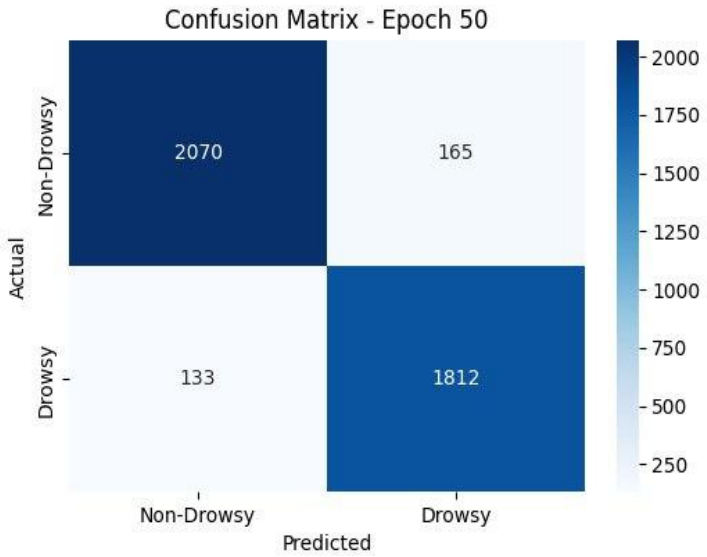
- Visual representations of the confusion matrix and ROC curve are provided for selected training epochs.
  - Confusion Matrix yielded from training of DL model for 10 iterations.

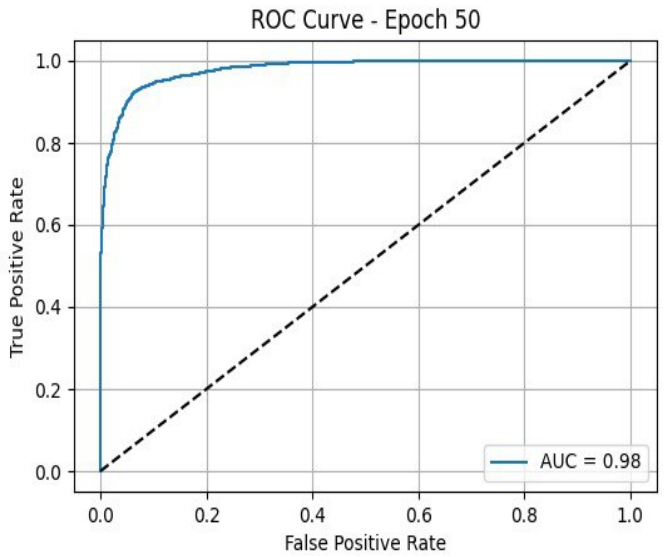




Binary Classification

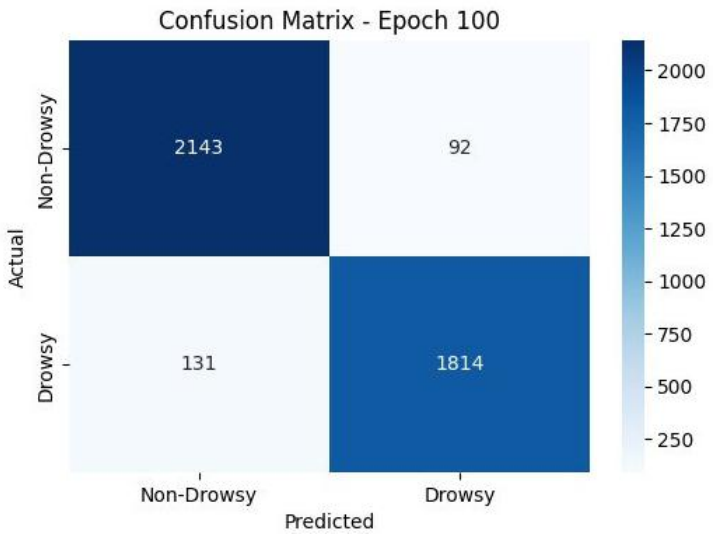
- Visual representations of the confusion matrix and ROC curve are provided for selected training epochs.
  - Confusion Matrix yielded from training of DL model for 50 iterations.

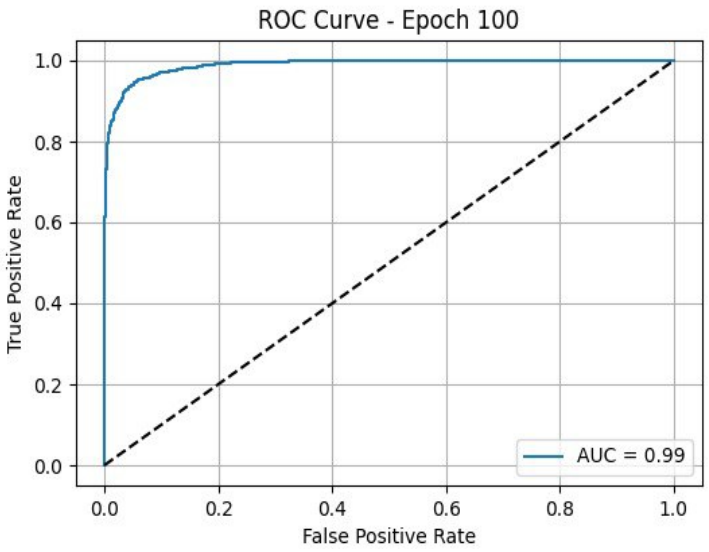




Binary Classification

- Visual representations of the confusion matrix and ROC curve are provided for selected training epochs.
  - Confusion Matrix yielded from training of DL model for 100 iterations.





Conclusion

- The project successfully created a **real-time** driver drowsiness detection system using the **Arduino Nicla Vision** and a **buzzer** to alert the driver. It aims **to improve road safety** by warning drivers when signs of drowsiness appear.
- The system uses a **small, AI-powered camera** that runs a **lightweight** model based on **MobileNetV2**. If the driver's eyes stay closed for more than 3 seconds, the buzzer sounds until the eyes open again.
- A dataset of **over 41,000 images** helped train the model to detect drowsiness accurately.
- **Testing** showed that the system works well in **real conditions** and can help reduce accident risks.

DRIVER DROWSINESS DETECTION SYSTEM

Evaluation

- A **binary classification** model was used to detect if the driver was drowsy or not based on eye status. The model was trained on over **41,000 images** and improved with each training stage.
- By the end of training, it reached **92.87% accuracy**, with **91.65% precision** and **93.16% recall**. The false negative rate dropped to **6.84%**, and the model achieved an **AUC of 0.98** and **F1-score of 0.92**.
- These results show that the system works well in **real-time** and is **reliable** for helping prevent drowsy driving accidents, even when running on a small device like Nicla Vision.

DRIVER DROWSINESS DETECTION SYSTEM

Future Works

- **Add GPS:** Include GPS to send alerts with the driver’s location in emergencies.
- **Improve the Model:** Use a larger dataset and test it with different people and lighting for better accuracy.
- **Support for Glasses:** Make the system work even if the driver is wearing glasses.
- **Battery Powered:** Allow the system to run on a battery for more flexibility.
- **Better Performance:** Keep improving the system to make it more accurate and reliable in all situations.

جامعة الأميرة نورة بنت عبدالرحمن
Princess Nourah bint Abdulrahman University

Thank You